FlowNova AI

A detailed product and engineering guide

Visual workflow automation, from draft to recorded execution



Scope and status

This guide describes the working Vercel-native hosted demo and the Java relational runtime. Planned capabilities are explicitly identified. It does not claim enterprise production readiness, benchmarked scale, or compliance certification.

Live app: https://flownova-ai.vercel.app
Source: https://github.com/manikantbindass/FlowNovaAI
Public demo login: demo / demo
Edition: 26 October 2026

Guide map

Use this guide as a product tour, an implementation reference, and a deployment starting point.

Part	Topics
Product	Purpose, users, walkthrough, visual editor, real execution
Architecture	Boundaries, data model, identity, versions, worker lifecycle
Behavior	Node reference, mapping and branches, AI boundaries, observability
Operations	Working Vercel demo, relational setup, testing, security
Future work	Scaling gates, 2027 proposals, standards and source references

Available today

Authenticated local workspaces; editable workflow graphs; server graph validation; explicit revision-safe saves; immutable publishing; SQL-backed execution jobs; node checkpoints; cancellation; history; real metrics; deterministic templates; optional configured AI. Advanced nodes add collection processing and assertions; version restore copies snapshots into draft revisions.

Still planned

Browser OIDC onboarding, signed webhooks, schedules, encrypted tenant credentials, safe HTTP/app connectors, retrieval with citations, billing, advanced agents, shared rate/session infrastructure, and performance validation.

The problem and the users

Repeated data handling often becomes a chain of manual copy/paste steps. FlowNova turns that chain into a graph whose behavior can be reviewed and recorded.

Who it serves

Developers preparing payloads, operators routing requests, support teams classifying incoming work, and engineering teams that need an auditable automation path. The present release is most appropriate for local evaluation and contributors, rather than unsupervised enterprise production use.

Current example

Supply a JSON message and priority. A manual trigger passes it to a condition. The true branch prepares an urgent request; the false branch prepares a standard request. The engine records completed and skipped nodes. No network side effect or AI key is required.

Why a visual graph

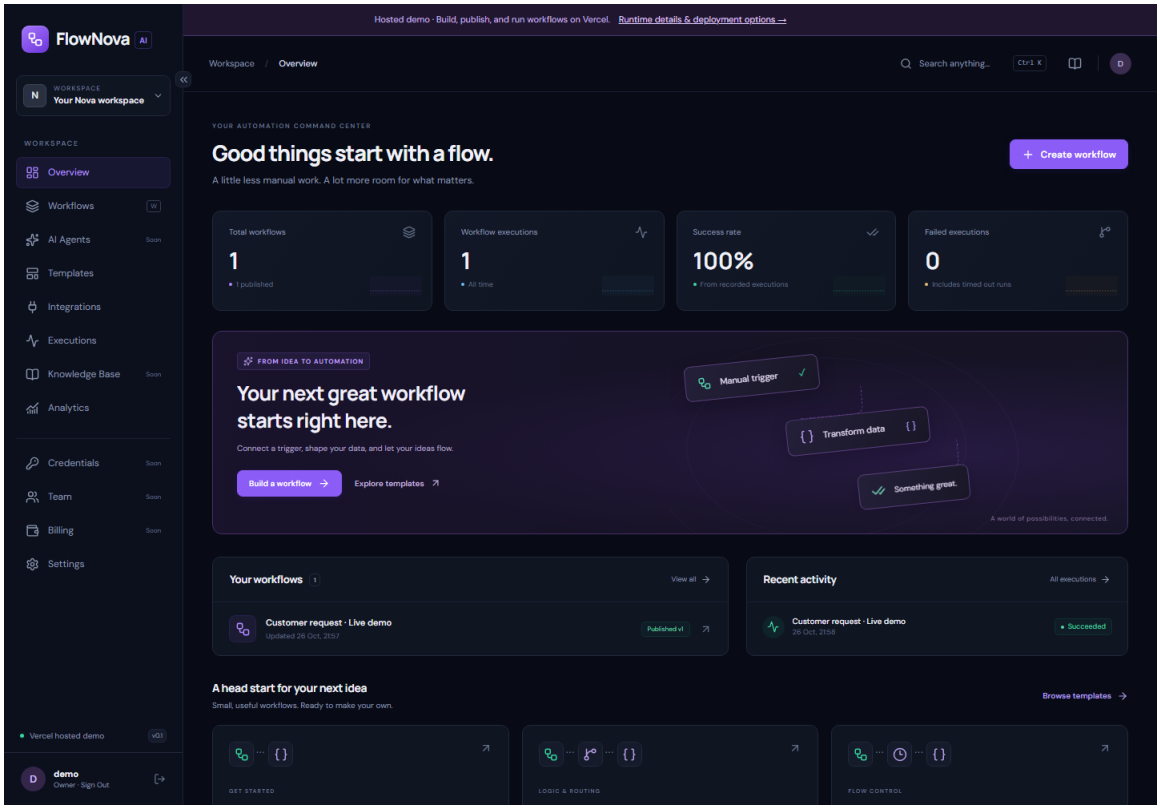
Nodes explain the steps, edges explain order and branches, and node configuration explains data handling. Publishing creates a reviewable snapshot, while execution history provides evidence of what actually ran.

What the product does not infer

A polished screen does not prove provider connectivity, security certification, or scalable throughput. Planned integration cards and roadmap pages do not represent working connectors.

Inside the workspace

The overview presents counts from the database and links to workflows, templates, and recorded executions.



Real local workspace screenshot. Records visible here were created during local evaluation or documentation capture.

A complete first journey

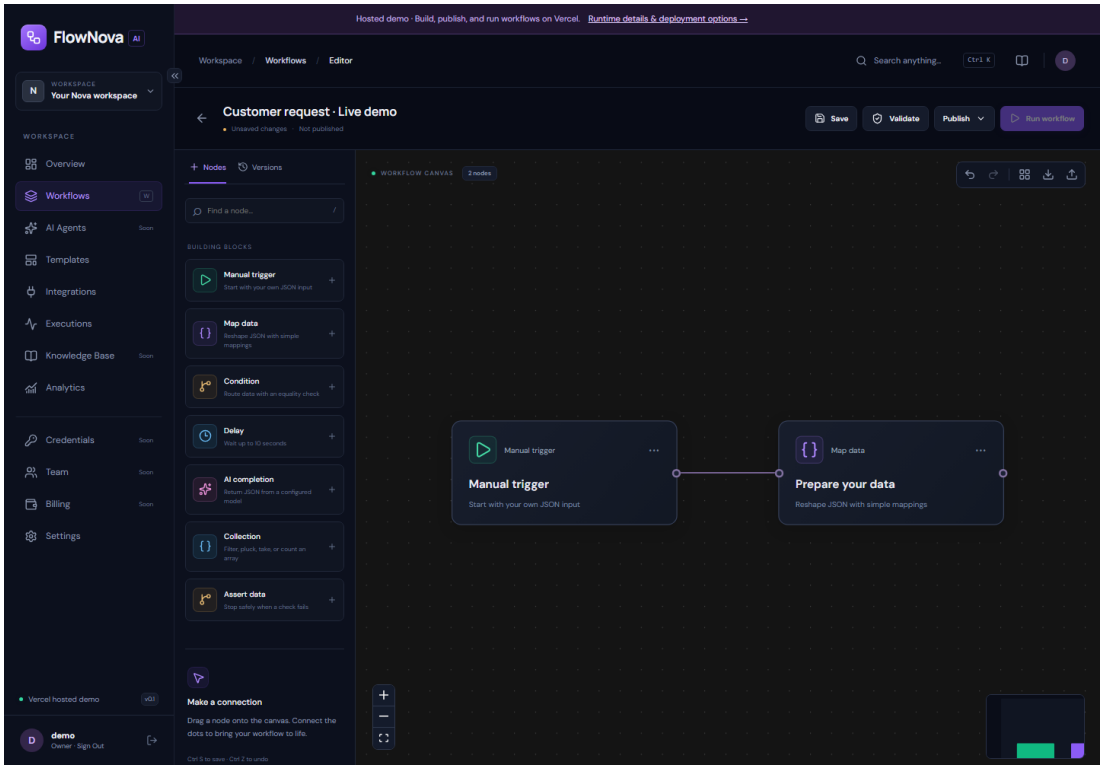
Sign in with demo / demo. Create a workflow or clone a template. Configure nodes, apply changes, save, validate, publish with confirmation, run with JSON input, and inspect the result. The viewer account can read the workspace but cannot write through the API.

Meaningful navigation

The application opens implemented workflow, template, execution, analytics, integration-status, and settings pages. Agents, credentials, knowledge, team management, and billing disclose their planned status.

The visual workflow editor

React Flow provides the graph surface; the Java API remains authoritative for validation, publication, and execution.

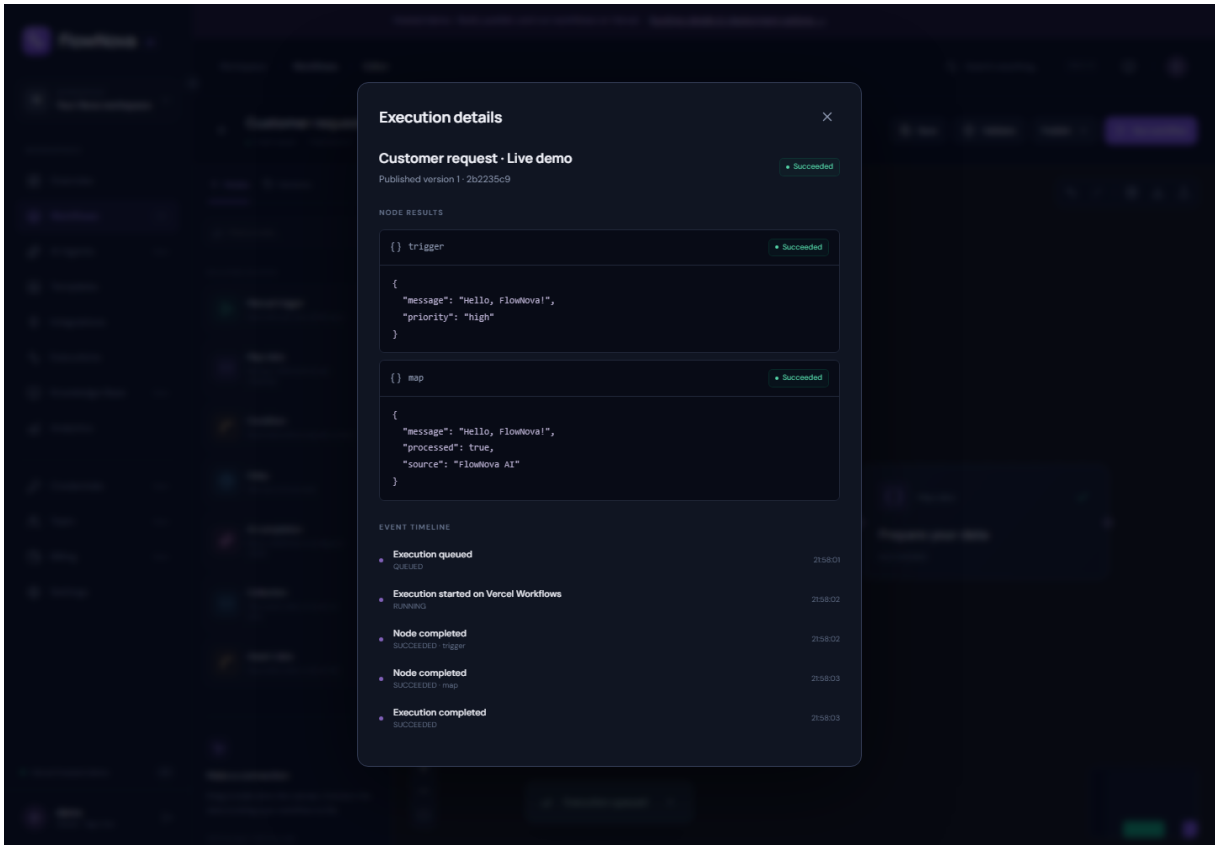


Editable graph with palette, connection handles, canvas controls, and draft state.

Control	Behavior
Palette and configuration	Add or drag nodes, connect handles, configure JSON, apply configuration
Editing	Move/select/delete, copy/paste, undo/redo, grid snapping, auto-layout
Persistence	Explicit Save or Ctrl S; revision conflicts are visible
Interchange	JSON export; schema-validated import; backend publish validation
History	Immutable versions; restore copies a snapshot into a new draft

A workflow that really runs

Run requests reference an exact published version and supply an object input. The browser observes database records rather than fabricated progress.



A real completed manual-trigger and mapping execution with persisted outputs and events.

Input and output

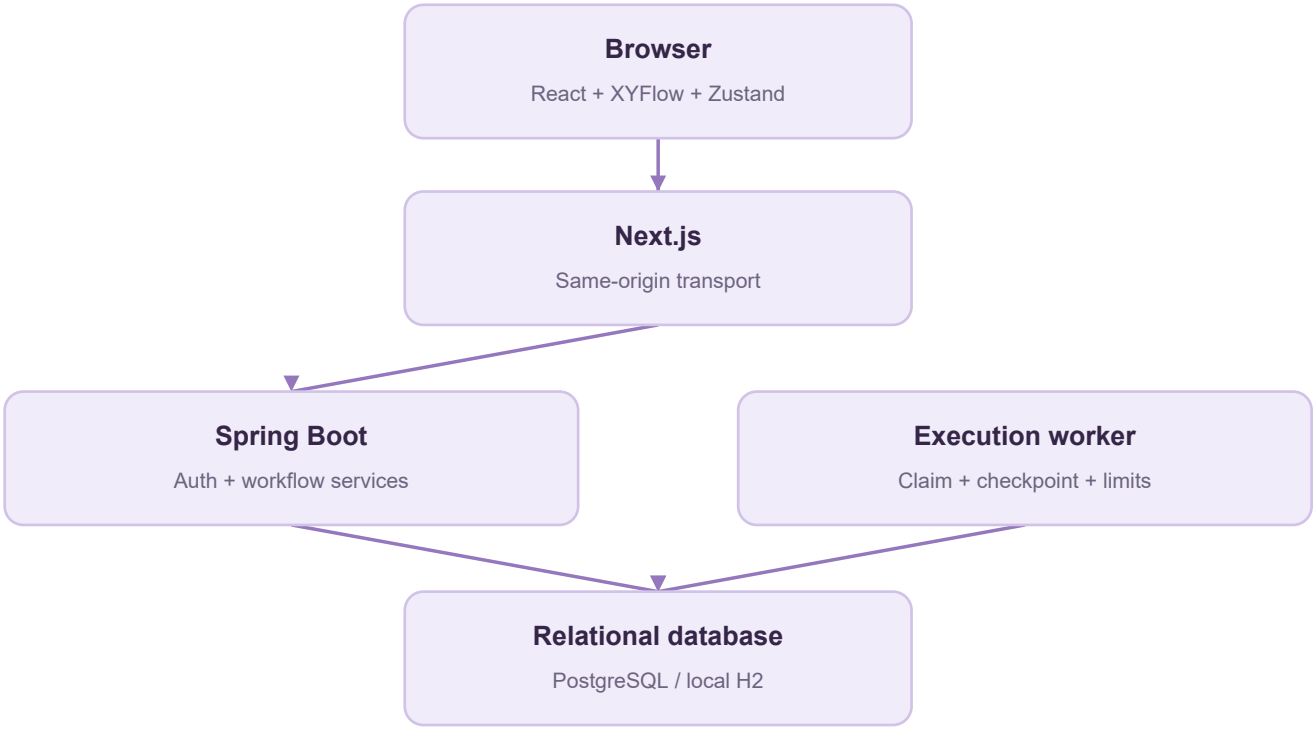
Example input: {"message":"Hello, FlowNova!","priority":"high"}. The starter mapping returns message, processed=true, and source="FlowNova AI". A later draft edit does not affect the already-published snapshot.

Cancellation and history

Eligible queued/running executions can be cancelled. Completed records remain inspectable in history. Polling refreshes details from persisted events and checkpoints; SSE/WebSocket reconnect transport is not implemented.

Architecture and ownership

A modular monolith keeps the first product understandable and avoids duplicating business rules in multiple runtimes.



Vercel-native runtime

Without API_ORIGIN, Vercel Functions implement the hosted API, iron-session seals browser-scoped workspace identity and CSRF data, private Blob documents persist definitions and results with conditional writes, and Vercel Workflows runs durable node steps and delays. No visitor Java setup is required. This is a bounded demo, not an unlimited relational service.

Frontend responsibility

Render the product and manage editor state and form feedback. With server-side API_ORIGIN configured, forward requests to the external Java API, preserving protected cookies and CSRF headers without exposing provider credentials.

Backend responsibility

Verify identity and membership, enforce roles and limits, validate graphs, save revisions, publish snapshots, queue jobs, persist execution state, record audit events, and compute metrics. The worker runs in the same Java process initially.

The data model

Definitions, immutable versions, and execution records have separate responsibilities.

The tables below describe the Java relational mode. The hosted adapter mirrors these concepts in a bounded private workspace document; cookie identity replaces the local membership seed, and Vercel Workflows owns durable orchestration.

Table	Ownership and purpose
workspace + membership	Workspace identity and actor role; every tenant operation verifies membership
workflow	Editable draft, revision, current published version, update actor and timestamp
workflow_version	Immutable graph snapshot identified by workflow/version and workspace
execution	Published version reference, status, input, claim/lease, attempt, idempotency key
node_execution	Per-node terminal checkpoint, output, start and finish time
execution_event	Persisted execution timeline
audit_log	Actor, workspace, action, resource, and timestamp

Constraints and transactions

Composite workspace/workflow foreign keys preserve ownership. Version and node-checkpoint keys prevent duplicate records. Workspace idempotency keys prevent duplicate queued runs. Draft saves use expected revisions; publication locks the workflow. Execution creation and queue event insertion share one database transaction.

Migration strategy

Flyway owns schema migrations. JSON graphs use TEXT for local H2/PostgreSQL portability. Do not rewrite applied migrations. Add forward-compatible migrations and test restores before production changes.

Identity, roles, and tenant isolation

The local account system is intentionally for development. The production profile is an external JWT resource server.

Mode	Current behavior
Hosted demo	iron-session sealed cookie, CSRF, private browser-scoped workspace; public demo login
Local Java sessions	Spring form login, BCrypt, HttpOnly/SameSite cookie, CSRF token/header
Production Java API	JWT issuer/audience validation; provisioned subject memberships
Roles	Owner/admin/editor can write; viewer cannot perform write operations
Tenant boundary	Authenticated workspace/role checks; inaccessible resources return 404

Public accounts

demo / demo opens the Nova workspace. viewer / demo has read-only membership there. other / demo belongs to a separate workspace. LOCAL_PASSWORD overrides the shared development password. Production disables these users.

Authorization is server-side

Hiding a button is not a security boundary. Tests directly invoke APIs/services to deny cross-tenant reads, changes, publication/execution/cancellation, and viewer writes.

Remaining identity work

Hosted browser OIDC login, registration, email verification, recovery, MFA, invitations, membership administration, and shared session infrastructure are not delivered by the current production resource-server configuration.

Drafts, versions, and safe restoration

The workflow lifecycle protects a run from accidental changes to its definition.

Step	Invariant
Save draft	Expected revision must match; stale writes receive 409
Validate	Type/configuration/topology checks run on the server
Publish	A new snapshot row is inserted; published history is not updated
Execute	The run references one workflow-version pair
Edit	Draft changes do not change prior versions
Restore	Copy an old graph to a new draft revision; publishing still requires confirmation

Concurrency

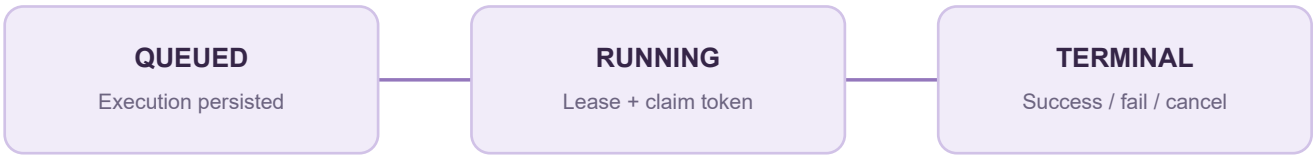
Two browser tabs may edit the same revision. The first successful save increments the revision. The second must reload or reconcile; it cannot silently overwrite. Real-time collaboration and automatic conflict merge are future features.

Import and export

Imports validate supported schema/version and size bounds in the browser. The backend still validates the graph before publication. Exports are useful for review and local backup; they are not a substitute for database backup.

Execution durability and recovery

In Java mode the SQL database doubles as a durable job queue. On Vercel, durable workflows and steps coordinate execution while private workspace records retain results.



Claim and checkpoint

A conditional UPDATE claims a queued execution. The worker refreshes a 60-second lease while checking status and the 120-second workflow deadline. Completed node outputs persist so eligible work can resume. Checkpoint commits lock the execution and verify the current claim token.

Recovery bounds

An expired running lease returns to the queue for bounded recovery. After three attempts it fails rather than staying RUNNING forever. Tests simulate stale leases and resume from an existing checkpoint; this is not equivalent to a validated distributed failover exercise.

Side-effect semantics

Pure deterministic nodes are safe to resume from checkpoints. A crash during a provider request can repeat an AI call and its cost. No exactly-once claim is made for arbitrary external APIs. The current worker is sequential per Java process.

Supported node reference

All supported nodes use schemaVersion=1. The backend rejects unsupported types and unbounded graph structure.

Node	Configuration and result
Manual	config={}; passes the execution input; exactly one per graph
Transform	mapping object; literals or $.path references; missing path becomes null
Condition	path + equals; returns matched and data; true/false output handles
Delay	milliseconds integer 0-10000; cancellation checks during wait
AI completion	prompt <=4000 characters; configured model; JSON object; 25s/512 output tokens
Collection	path + operation; filter/pluck/take/count; <=1000 inspected, <=100 returned
Assert	path + equals; mismatch fails with a safe, fixed error

Graph bounds

One to 50 nodes and at most 100 edges; one manual trigger; all nodes reachable; no cycles; one incoming edge per non-trigger node; one edge per output handle; no ambiguous joins.

Data mapping and branching

Data flows from each node into the selected downstream node. This is a deliberately small expression system, not arbitrary JavaScript.

```
{"mapping": {"customer": "$.customer.name", "processed": true}}
```

Literal versus reference

A string beginning with $. reads a dot-separated path from the previous node output. Other strings, booleans, numbers, arrays and objects are literal values. Missing paths become JSON null. Mapping keys are bounded.

```
{"path": "$.priority", "equals": "high"}
```

Condition output

A condition returns {matched:true|false,data:originalInput}. Connect true and false handles. A downstream mapping that needs the original message reads $.data.message. Nodes on an unselected branch are recorded SKIPPED, and that state propagates through descendants.

Collections and assertions

Collection operations return {items,total,returned}; count omits item output. Assert stops the run when required data is absent or different. Neither node executes a script or accesses a network service.

AI without hidden autonomy

The present AI node is a bounded provider call. It is not a self-directed agent and has no tool or credential discovery.

Control	Current boundary
Provider	OpenAI adapter; server key; server-configured model
Input	Prompt plus field-redacted JSON; no browser key storage
Output	JSON object mode; object parsing; maximum 512 output tokens
Time	5-second connection timeout; 25-second request timeout
Failures	Missing credentials and HTTP/provider errors become understandable failures
Tools	None; no network tools, delegated agents, or automatic side effects

JSON validity is not semantic correctness

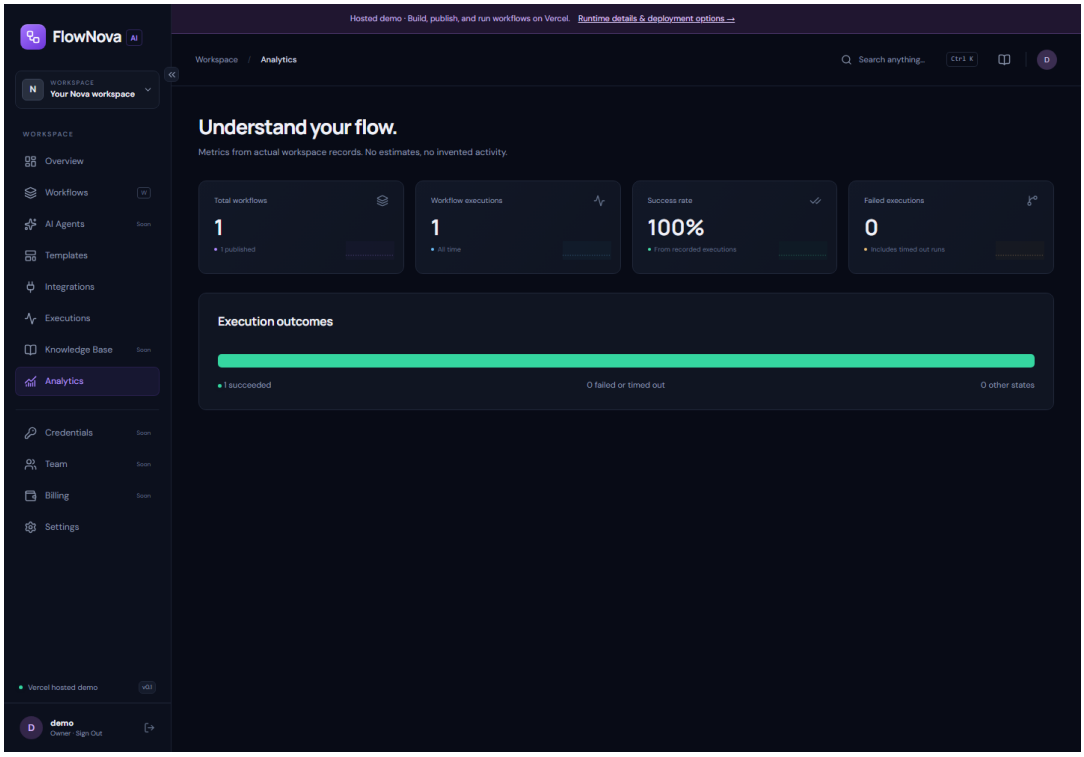
The current adapter checks an object response. It does not guarantee a caller-defined JSON Schema or factual correctness. Schema-constrained generation, refusal/truncation handling, provider mocks, evaluations, and usage ledgers are planned.

Costs and testing

Only configure a provider key when real provider calls and their charges are intended. No live AI request was required for the deterministic local flow. A configured flag indicates key presence, not a test-connection success.

Observability and real metrics

Execution details show persisted node results and an event timeline. Analytics computes counts from workspace records.



Workspace metrics derived from actual database records; no fabricated historical series.

What is measured

Workflow count, published workflow count, execution count, succeeded count, failed/timed-out count, and all-run success percentage. Queued/running/cancelled records remain in the denominator; the interface does not invent a success trend.

What comes next

OpenTelemetry spans through API/worker/provider, queue-depth and queue-age gauges, token/cost ledgers, SLOs derived from measured workloads, retention, structured log correlation, and reconnectable event cursors.

Local startup and configuration

The complete deterministic product runs without a paid provider or cloud database.

```
npm ci
# API terminal
$env:JAVA_HOME = 'C:\Program Files\Java\jdk-25'
$env:SERVER_PORT = '8181'
cd apps/api
.\mvnw.cmd spring-boot:run

# Web terminal, repository root
$env:API_ORIGIN = 'http://127.0.0.1:8181'
npm run dev --workspace apps/web -- --port 3210
```

Prerequisites and login

Node 22+ and JDK 21+; the inspected machine uses Node 24 and JDK 25. The Maven wrapper downloads Maven. Open http://127.0.0.1:3210 and sign in with demo / demo. Select the real installed JDK path; the example is Windows-specific.

Persistence and optional Docker

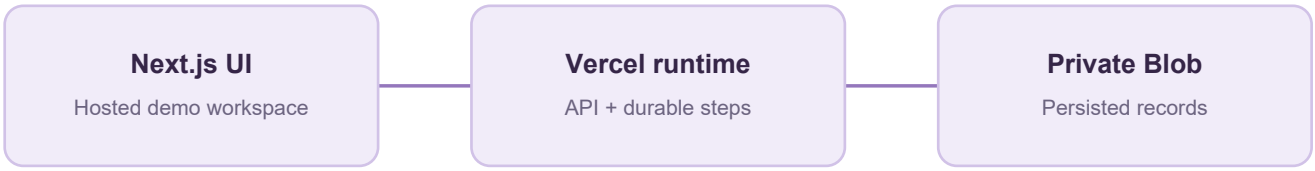
The local H2 profile stores files in apps/api/.data. Compose configures PostgreSQL, API and web on loopback ports 5432/8080/3000. The original Docker daemon was stopped, so container startup was not verified there. Host Maven does not automatically load a .env file.

Troubleshooting

Check API health, API_ORIGIN and occupied ports. Do not stop unrelated applications. Avoid running a build against a cache directory being served by another process. Synchronized Windows folders can retain generated symlink/cache locks; separate NEXT_DIST_DIR output can isolate builds.

Working hosting on Vercel

The public deployment includes the hosted API, protected sessions, private workspace persistence, and durable workflow execution. Visitors can build and execute without configuring a separate backend.



Hosted runtime

https://flownova-ai.vercel.app uses Next.js Functions, iron-session, a connected private Vercel Blob store, and Vercel Workflows. API_ORIGIN remains unset. Public demo/demo credentials create a private browser-scoped workspace. Clearing cookies loses its reference. Limits: 25 workflows, 20 versions each, 50 run records, two active runs, 2 MB workspace data, and Vercel Hobby provider caps.

Relational deployment option

For the Java mode, host Java and PostgreSQL separately; configure production profile, database URL/user/password, OIDC issuer/audience and memberships; terminate TLS; configure secrets, backups, limits and monitoring; set server-only API_ORIGIN in Vercel; redeploy. Verified browser OIDC onboarding remains additional work. The existing Vercel demo does not require this setup.

Production controls

Replace the public development accounts, test tenancy against PostgreSQL, establish compatible migration rollouts, verify backup restoration, define retention and incident procedures, and approve any paid resource provisioning separately.

Security controls and remaining gaps

Security boundaries should be assessed by behavior and tests, not by the appearance of the product.

Implemented boundary	Important limitation
Membership + roles + scoped queries	RLS and broader production policy still need assessment
CSRF + protected local cookies	Shared sessions and browser OIDC are not implemented
Request/runtime/output caps	Per-IP request rate limits are process-local
Field-name redaction	Not general DLP; do not embed secrets in arbitrary graph strings
No HTTP or script node	Network egress and code sandboxing must be designed before adding them
Bounded AI without tools	No guarantee of factual/semantic output correctness

Data retention and credentials

Source graphs are stored as supplied. Tenant credential encryption, secret references, deletion/retention policies, malware scanning, and document storage are future work. Local demo records must not contain private production data.

Supply chain and deployment

CI defines dependency, source-secret, and container scans. Remote scan success is distinct from local test success. Use least-privilege roles and externally managed secrets, and verify TLS and deployment access before production use.

Testing and release acceptance

The release gate requires behavior evidence across the browser, Java services, and database contracts.

Layer	Checks
Frontend	TypeScript, ESLint, interchange-contract tests, production build
Java	Graph validation, saves/revisions, publication, branches, node processing, restore
Security	Unauthenticated/CSRF denial, viewer write denial, cross-tenant access denial
Execution	Idempotency, cancellation, deadline, stale lease recovery and checkpoints
Browser	Create, save, validate, publish, run, inspect, reload and mobile layout

Use the source progress record

The repository docs/PROGRESS.md records final commands and results. Tests run locally do not prove production scaling or arbitrary provider reliability. The default Java suite uses H2; CI includes a disposable PostgreSQL service test.

Required before production

Actual worker-process kill/restart, PostgreSQL concurrent claims, high-load fairness, external identity, provider refusal/rate-limit mocks, backup restore, retention/deletion, and dependency/container remediation. No measured throughput or exactly-once guarantee is asserted.

Scaling the platform deliberately

The initial SQL queue and module boundaries allow future growth, but scaling must follow measurement.

Separate workloads

The Java path can introduce explicit API and worker roles and bounded pools, measured using queue age/depth and database saturation. The Vercel adapter is a deliberate response to the request for a no-setup hosted core; preserve node semantics with parity tests across both runtimes. Move beyond a bounded Blob document only when workload evidence supports it.

Protect tenants and dependencies

Add per-tenant concurrency and budgets, shared rate limits, backpressure, circuit breakers, bounded retries with jitter, provider-specific side-effect rules, and cancellation propagation. Test noisy-neighbor behavior and stale-worker fencing.

Operational targets, not promises

A controlled 100-active-user workload and 1,000 queued jobs are proposed targets. Report hardware, payload size, latency percentiles, error rates, queue drain rate and bottlenecks. Add Redis or object storage only when a concrete workload requires them; avoid premature Kubernetes.

Retention and recoverability

Partition/archive high-volume event and usage records when measurements justify it. Establish backup schedules, restore drills, schema compatibility and incident playbooks before using production reliability language.

2027 proposals and references

The future roadmap extends the tested core with controlled tools and agents. These items are proposals, not delivered features or predictions.

Phase	Proposed scope and gate
Hosted core	OIDC onboarding, shared limits/sessions, worker roles, PostgreSQL load and backup evidence
Connectors	Encrypted credentials, safe HTTP egress, signed webhooks, schedules, GitHub/Slack tests
Agents	Tool allowlists, approval gates, budgets, schema-constrained generation and evaluations
Knowledge/governance	Safe document ingestion, tenant retrieval, citations, deletion, audit retention, billing reconciliation

Interoperability

Evaluate MCP for approved tool/resource access with consent and isolation. Evaluate A2A for authenticated delegation while preserving identity, cancellation, provenance and budgets. Neither protocol is integrated in this release.

Primary references, consulted 26 October 2026
MCP architecture: https://modelcontextprotocol.io/specification/2026-07-28/architecture
A2A specification: https://a2a-protocol.org/latest/specification/
OpenAI structured output: https://developers.openai.com/api/docs/guides/structured-outputs
Repository source and architecture.md remain the authority for implemented behavior.